ENTITY FRAMEWORK

Для подключения и работы с базой данных через приложение используется технология ADO.NET — это набор классов, встроенных прямо в .NET Framework и .NET Core, которая позволяет программе на C# взаимодействовать с базой данных. Для удобства взаимодействия существует как минимум 2 способа:

1. MySQL.Data

MySql.Data — это официальный ADO.NET - драйвер для подключения приложений на платформе .NET к базе данных MySQL, разрабатываемый непосредственно корпорацией Oracle. Данная технология является первым и самым распространенным способом связки C# и MySQL.

2. Entity Framework

Entity Framework предлагает иной путь: работа с базой данных происходит через набор C#-объектов. Вы просто меняете свойства объектов и добавляете их в списки, а EF сам превращает эти действия в SQL-запросы типа INSERT, SELECT, UPDATE и выполняет их через ADO.NET.

Entity Framework (EF) — это объектно-ориентированная технология доступа к данным (ORM, Object-Relational Mapping), разработанная Microsoft. Она позволяет разработчикам работать с базами данных, используя объекты и классы C#, вместо написания SQL-запросов вручную. Entity Framework автоматически преобразует операции с объектами в SQL-запросы и обратно, что значительно упрощает взаимодействие с базой данных.

Проще говоря, EF позволяет:

- Описывать структуру таблиц как C#-классы;
- Выполнять запросы к БД через LINQ;
- Автоматически создавать и обновлять схему базы данных.

ORM — это подход, который связывает классы и объекты языка программирования с таблицами и строками в базе данных.

SQL	C#
Таблица Users	Класс Users
Поле Name	Свойство Name
Строка в таблице	Объект класса

Для современных приложений используется Entity Framework Core (EF Core) — кроссплатформенная, лёгкая и расширяемая версия EF.

Преимущества EF

- Работа с БД через C#-код;
- Поддержка миграций (изменений схемы);
- Интеграция с LINQ;
- Кроссплатформенность (в EF Core);
- Поддержка транзакций, отложенной загрузки, валидации.

Основные компоненты:

- **DbContext**

Контекст подключения к БД и управления сущностями, главный класс для связи и взаимодействия с БД. Он хранит в себе строку подключения, а также управляет открытием и закрытием подключения к БД. Класс также содержит все таблицы, с которыми работает программа и отслеживает все изменения, произошедшие с ними.

Пример класса:

```
public class ApplicationDBContext : DbContext
   {
      public DbSet<Patients> patients { get; set; } = null!;

      protected override void OnModelCreating(ModelBuilder modelBuilder)
      {
         modelBuilder.Entity<Patients>(patientsConfiguration =>
         { patientsConfiguration.HasKey(p => p.id_patient);
         });
      }
      protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
      {
         optionsBuilder.UseMySql(
            "server=localhost; user=root; password=123; database=hospital;", new MySqlServerVersion(new Version(8, 0, 41)));
      }
   }
```

Использование в программе:

```
using var db = new ApplicationDBContext();
var patients = db.Patients.ToList();
```

– **DbSet<T>**

Объект внутри контекста, представляющий конкретную таблицу в базе данных. С помощью DbSet можно получать, добавлять и удалять данные.

– **Классы-сущности (Entity)**

Описывают таблицы БД через C#-модель, это обычный класс C#, который описывает, какие данные хранятся в таблице. Каждое свойство класса соответствует колонке в таблице, каждый объект класса — одной строке в таблице.

– **Конфигурация модели**

Способ объяснить Entity Framework, как именно связать наши классы с таблицами в базе данных. Обычно EF сам догадывается по именам классов и свойств, какая таблица какому классу соответствует. Но если база уже существует и названия в ней отличаются от тех, что в коде, или нужно задать дополнительные правила.

В Entity Framework существует три подхода к работе с базой данных: Code First, Database First и Model First. Они отличаются тем, с чего начинается разработка — с кода, с базы данных или с визуальной модели.

– **Code First**

Подход Code First означает, что разработка начинается с написания кода на C#. Сначала создаются обычные классы, описывается контекст данных, настраиваются связи. Базы данных на этом этапе ещё нет. Entity Framework на основе созданных классов сам генерирует структуру базы данных — таблицы, ключи, связи, индексы. Разработчик вообще не пишет SQL, а только работает с объектами.

Для переноса классов в таблицы БД используются миграции — механизм, который позволяет синхронизировать структуру базы данных с изменяющейся моделью классов. Когда проект развивается, классы меняются: добавляются новые свойства, удаляются старые, меняются типы. База данных должна соответствовать этим изменениям. Миграции автоматически фиксируют, что именно изменилось, и создают код для обновления структуры базы без потери данных.

Процесс состоит из двух шагов. Сначала разработчик вносит изменения в классы. Затем создаётся миграция — специальный файл, в котором EF фиксирует, что именно изменилось по сравнению с предыдущей версией. Этот файл содержит код на C#, который описывает необходимые изменения в базе: создание новых таблиц, добавление колонок, изменение типов.

После создания миграции её нужно применить к базе данных. Команда обновления выполняет сгенерированный код, и структура базы приводится в соответствие с текущей моделью. Если изменений несколько, они применяются последовательно.

Например, создадим классы

```
public class Playlist
{
    public int Id { get; set; }
    public string? Name { get; set; }
    public DateTime CreatedAt { get; set; }


    public List<Track>? Tracks { get; set; }
}


public class Track
{
    public int Id { get; set; }
    public string? Title { get; set; }
    public string? Artist { get; set; }
    public string? Album { get; set; }
    public int Duration { get; set; }
    public DateTime AddedAt { get; set; }


    public int PlaylistId { get; set; }
    public Playlist Playlist { get; set; } = null!;
}
```

Создадим еще один класс для контекста базы данных, наш класс наследует DbContext.

```
public class MusicDbContext : DbContext
{
}
```

Пропишем внутри класса DbSet

```
public DbSet<Playlist> Playlists { get; set; }
public DbSet<Track> Tracks { get; set; }
```

Также переопределим метод OnConfiguring, создающий подключение к БД

```
protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
{
   var connectionString =
"server=localhost;database=MusicDb;user=root;password=1234";
   optionsBuilder.UseMySql(connectionString, new MySqlServerVersion(new
Version(8,0,41)));
}
```

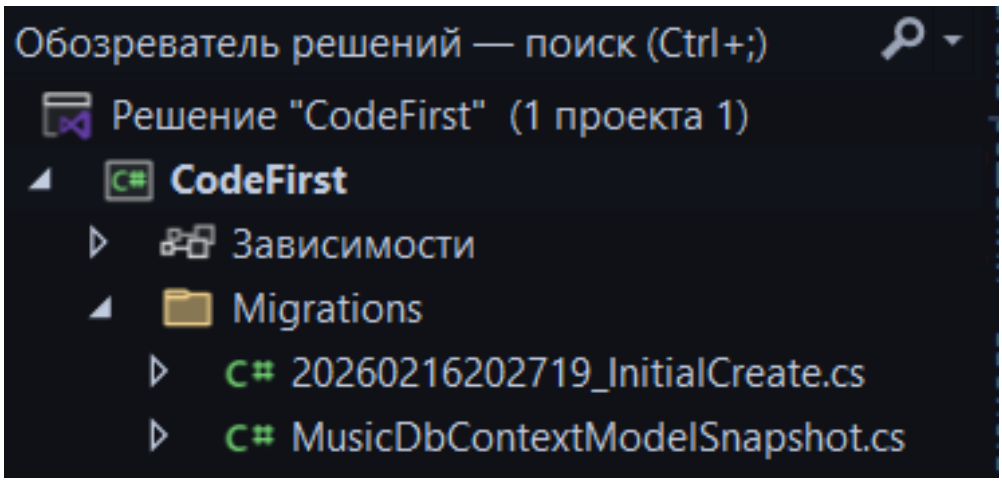
Теперь переопределим метод OnModelCreating, где происходит настройка таблицы, установим внешний ключ

```
protected override void OnModelCreating(ModelBuilder modelBuilder)
{
   modelBuilder.Entity<Playlist>()
       .HasMany(p => p.Tracks)
       .WithOne(t => t.Playlist)
       .HasForeignKey(t => t.PlaylistId);
}
```

Таблицы созданы, теперь можно перенести классы в таблицы БД. Для этого необходимо открыть PowerShell, который встроен в Visual Studio, и прописать команды

```
dotnet ef migrations add Название_миграции – создание миграции базы данных
dotnet ef database update – применение последней созданной миграции к базе
```

После успешной миграции в обозревателе решений появятся папка с двумя файлами

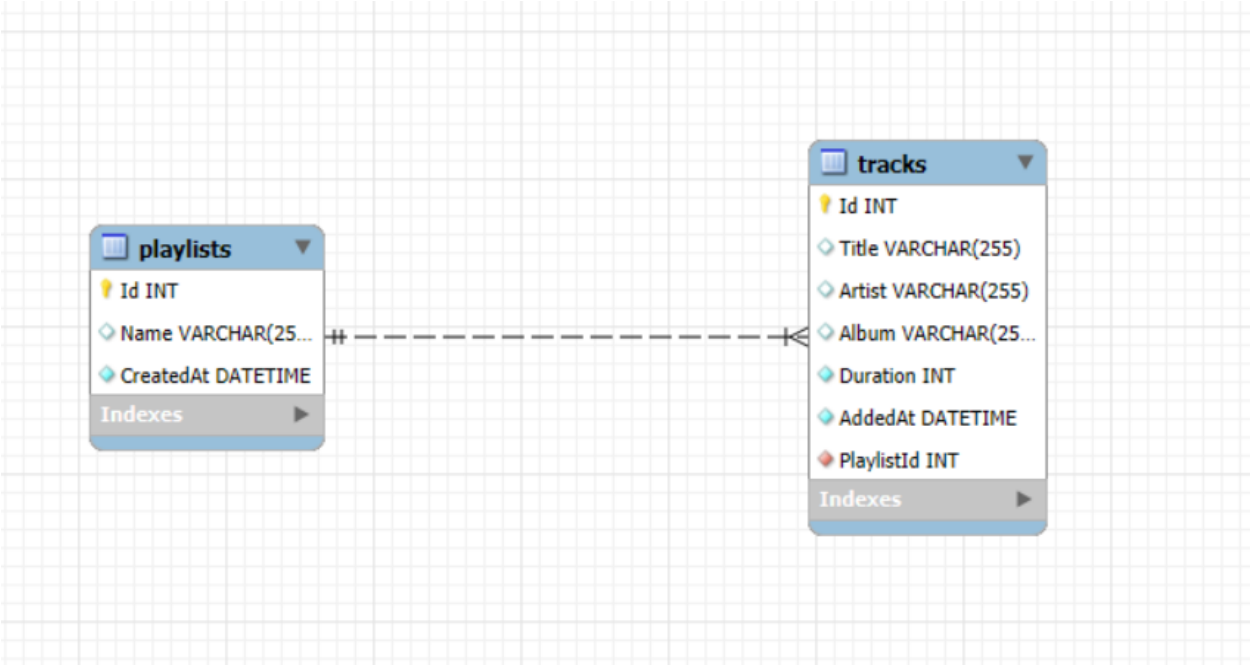


– **Database First**

Подход Database First работает в обратную сторону: сначала создается база данных, а код генерируется на основе её структуры. Entity Framework подключается к готовой базе, анализирует таблицы, ключи, связи и типы данных, а затем автоматически создаёт классы-сущности и контекст данных, соответствующие этой структуре.

Перенос БД производится с помощью скаффолдинга (Scaffold) — метод автоматизированной генерации базовой структуры проекта или программного кода на основе моделей данных. В результате скаффолдинга получается полный набор необходимых компонентов: классы-сущности для каждой таблицы, класс контекста с настроенными DbSet, навигационные свойства для всех связей между таблицами. Всё это готово к использованию сразу после генерации.

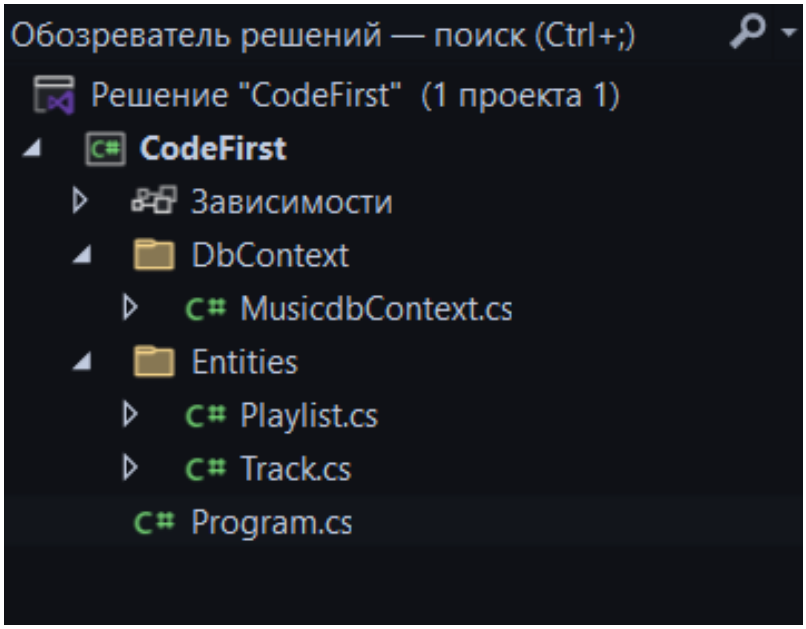
Рассмотрим пример. У нас есть база данных



Теперь добавим ее в проект с помощью команды

dotnet ef dbcontext scaffold "строка соединения с базой данных" Pomelo.EntityFrameworkCore.MySql --context-dir DbContexts --output-dir Entities

После выполнения в обозревателе решений появятся 2 папки: контекст БД и сущности



Когда данные получены из базы и превращены в объекты, с ними нужно производить действия: искать нужные, фильтровать, сортировать, объединять. В Entity Framework для этого используется LINQ — Language Integrated Query, то есть запросы, встроенные прямо в язык C#.

Особенность LINQ в том, что он не выполняет запрос мгновенно. Когда вы пишете db.Products.Where(p => p.Price > 1000), вы не загружаете все товары, а потом фильтруете их в памяти. Вы создаёте запрос, который будет переведён в SQL и выполнен на стороне базы данных. Сам запрос уходит в базу только тогда, когда вы пытаетесь получить результат — например, вызываете ToList() или проходите по коллекции в цикле.

Основные команды

Where — фильтрация

Самая часто используемая команда. Она позволяет отобрать только те записи, которые соответствуют условию. Всё, что написано в Where, превращается в SQL-оператор WHERE и выполняется на сервере базы данных, а не в памяти приложения.

```
var shortSongs = db.Tracks.Where(t => t.Duration < 180).ToList();
```

Select — выборка определённых полей

По умолчанию EF возвращает все поля таблицы. Но иногда нужны не все данные, а только некоторые. Select позволяет спроектировать результат в нужную форму — либо анонимный объект, либо другой класс.

```
var songsNames = db.Tracks.Select(t => t.Title).ToList();
```

OrderBy — сортировка

Упорядочивает результат по возрастанию или убыванию.

```
var sortedTracks = db.Tracks.OrderBy(t => t.Duration).ToList(); // по возрастанию

var sortedTracksDesc = db.Tracks.OrderByDescending(t => t.Duration).ToList(); // по убыванию
```

Include — загрузка связей

По умолчанию EF не загружает связанные объекты — только данные из основной таблицы. Это сделано для производительности, чтобы не брать из базы лишнего. Если нужны связанные данные (например, товары вместе с их категориями), нужно явно указать это через Include.

```
var playlistsWithTracks = db.Playlists
    .Include(p => p.Tracks)
    .ToList();
```

FirstOrDefault — получить один объект

Используется, когда нужна только первая запись, удовлетворяющая условию, или нужно найти запись по идентификатору. Возвращает либо найденный объект, либо null, если ничего не нашлось.

```
var shortTrack = db.Tracks.FirstOrDefault(t => t.Duration = 175);
```

Консоль отладки Microsoft Visual Studio

```
Фильтрация

Friesenjung - Joost Klein       170
Ome Robert - Joost Klein        175
Glassie Water - Joost Klein     175

Выборка

Friesenjung
Europapa
Droom Groot
Wachtmuziek
Florida 2009
Ik Wil Je
Disco Belgica
Ome Robert
Antwoord
Meeuw
Door De Kerk
Demonen In Me Lichaam
Vogels
Glassie Water

Сортировка

Friesenjung - 170
Ome Robert - 175
Glassie Water - 175

Demonen In Me Lichaam - 230
Meeuw - 220
Wachtmuziek - 210

Связанные таблицы

Мой плейлист (треков: 4)
  Europapa - Joost Klein
  Antwoord - Joost Klein
  Droom Groot - Joost Klein
Joost Klein Collection (треков: 10)
  Friesenjung - Joost Klein
  Ome Robert - Joost Klein
  Glassie Water - Joost Klein

Поиск объекта

 Ome Robert - Joost Klein
```

EF в WPF

WPF работает с коллекциями, которые умеют уведомлять интерфейс об изменениях, а Entity Framework даёт данные в виде обычных списков.

Entity Framework возвращает данные в виде List<T> или IQueryable<T>. Это обычные коллекции, которые не сообщают интерфейсу о том, что в них что-то поменялось. WPF же для автоматического обновления экрана требует коллекции, реализующие интерфейс INotifyCollectionChanged — например, ObservableCollection<T>.

Если просто привязать List<T> к списку в интерфейсе, данные покажутся, но, если потом добавить новый товар в список, интерфейс об этом не узнает и не отобразит его. Пользователь увидит устаревшие данные.

Пример из приложения

```
private ObservableCollection<Playlist> _playlists = new
ObservableCollection<Playlist>();
private ObservableCollection<Track> _tracks = new
ObservableCollection<Track>();
```

CRUD

Аббревиатура CRUD расшифровывается как Create, Read, Update, Delete — создание, чтение, обновление и удаление. Это четыре основные операции, которые выполняет любое приложение, работающее с данными. Entity Framework делает эти операции максимально простыми и единообразными.

Read — чтение

Это получение данных из базы через запросы, фильтрацию и сортировку. Всё строится вокруг DbSet и LINQ.

Create — добавление новых записей

Чтобы добавить новую запись в базу данных, нужно создать объект соответствующего класса, добавить его в DbSet и вызвать SaveChanges.

```
using (var db = new AppDbContext())
{
    var newProduct = new Product
    {
        Name = "Новый товар",
        Price = 1500,
        CategoryId = 3
    };

    db.Products.Add(newProduct);

    db.SaveChanges();
}
```

После SaveChanges объект newProduct получит сгенерированный базой идентификатор — свойство Id заполнится автоматически.

Можно добавить несколько объектов перед одним SaveChanges — тогда все они сохранятся в одной транзакции.

Update — обновление существующих записей

Для обновления нужно сначала получить объект из базы, изменить его свойства и снова вызвать SaveChanges. Entity Framework сам отследит, что именно изменилось, и сгенерирует соответствующий UPDATE.

```
using (var db = new AppDbContext())
{
   var product = db.Products.Find(5);

   product.Name = "Новое название";
   product.Price = 2000;

   db.SaveChanges();
}
```

Delete — удаление записей

Для удаления объект нужно передать в метод Remove или RemoveRange.

```
using (var db = new AppDbContext())
{
       var product = db.Products.Find(5);

   db.Products.Remove(product);

   db.SaveChanges();
}
```